
Prof. Ryan Cotterell, Prof. Mrinmaya Sachan, Prof. Florian Tramèr

Assignment 1

Goerkem Kadir Solun, 25-936-154

Collaborators

1. Collaborator 1 Name, Collaborator 1 Legi

Declaration

I, Goerkem Kadir Solun, confirm that I have written this assignment independently, without any direct copying or paraphrasing from other students' work. I have acknowledged everyone I consulted during my research. I understand that while consultation and comparison of solutions are permitted, the final written submission must be my own work.

I am aware that any form of plagiarism is a violation of academic integrity. If it is determined that I have engaged in academic dishonesty, I accept and understand that the university's standard measures for plagiarism will be applied.

By submitting this assignment, I affirm my commitment to academic honesty and integrity.

Question 1: Representation Surgery

a) Linear Guarding vs. Arbitrary Classifiers

No. By Definition 2, the statement that $\mathbf{X}$ *linearly guards* Z only guarantees that the trivially attainable loss L_τ is optimal over the set of all **affine transformations** (linear predictors). Mathematically, it ensures that $\mathbb{E}[\mathcal{L}(f(\mathbf{X}), Z)] \geq L_\tau$ holds for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$. However, the classifier $c : \mathbf{x} \mapsto c(\mathbf{x}) \in \mathcal{Z}$ proposed in the question is an *arbitrary* classifier, meaning it is not restricted to being linear. Because linear guarding only constrains linear probes, a more complex, non-linear classifier such as a multi-layer perceptron could potentially draw non-linear decision boundaries to extract the concept information Z still embedded within $\mathbf{X}$. Consequently, such a non-linear classifier could successfully achieve a loss lower than the trivially attainable loss L_τ .

b) Class-Conditional Mean Equality Implies Linear Guarding

If each class-conditional mean satisfies $\mathbb{E}[\mathbf{X} \mid Z = i] = \mathbb{E}[\mathbf{X}]$, then $\mathbf{X}$ linearly guards Z with respect to the cross-entropy loss $\mathcal{L}$. To prove that $\mathbf{X}$ linearly guards Z , we must show that for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ (where $\mathbf{A} \in \mathbb{R}^{k \times d}$ and $\mathbf{b} \in \mathbb{R}^k$), the expected loss is bounded by the trivially attainable loss L_τ :

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] \geq L_\tau \quad (1)$$

We can rewrite the expected loss by conditioning on the class Z :

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] = \mathbb{E}_Z \left[\mathbb{E}_{\mathbf{X}|Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z) \mid Z] \right] \quad (2)$$

The cross-entropy loss $\mathcal{L}(\mathbf{y}, Z = i) = -\ln \frac{\exp(y_i)}{\sum_{j=1}^k \exp(y_j)}$ is convex with respect to its input logits $\mathbf{y}$. Because $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$ is an affine, the composition $\mathcal{L}(\mathbf{A}\mathbf{x} + \mathbf{b}, Z = i)$ remains convex with respect to $\mathbf{x}$. Applying Jensen's inequality gives:

$$\mathbb{E}_{\mathbf{X}|Z=i} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, i)] \geq \mathcal{L}(\mathbb{E}_{\mathbf{X}|Z=i} [\mathbf{A}\mathbf{X} + \mathbf{b}], i) = \mathcal{L}(\mathbf{A} \mathbb{E}[\mathbf{X} \mid Z = i] + \mathbf{b}, i) \quad (3)$$

Use the fact that class-conditional means equal the global mean, $\mathbb{E}[\mathbf{X} \mid Z = i] = \mathbb{E}[\mathbf{X}]$.

$$\mathbb{E}_{\mathbf{X}|Z=i} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, i)] \geq \mathcal{L}(\mathbf{A} \mathbb{E}[\mathbf{X}] + \mathbf{b}, i) \quad (4)$$

Substituting this lower bound back into the outer expectation over Z gives:

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(\mathbf{A}\mathbf{X} + \mathbf{b}, Z)] \geq \mathbb{E}_Z [\mathcal{L}(\mathbf{A} \mathbb{E}[\mathbf{X}] + \mathbf{b}, Z)] \quad (5)$$

Notice that $\mathbf{b}' \triangleq \mathbf{A} \mathbb{E}[\mathbf{X}] + \mathbf{b}$ is simply a constant vector in $\mathbb{R}^k$, independent of $\mathbf{X}$. Thus, the right side of the inequality evaluates the loss of a *constant* predictor $f(\mathbf{x}) = \mathbf{b}'$.

The trivially attainable loss L_τ is the infimum of the expected loss over all constant predictors:

$$L_\tau = \inf_{\mathbf{c} \in \mathbb{R}^k} \mathbb{E}_Z [\mathcal{L}(\mathbf{c}, Z)] \quad (6)$$

Since $\mathbf{b}'$ is a constant in $\mathbb{R}^k$, its expected loss must be greater than or equal to the infimum:

$$\mathbb{E}_Z [\mathcal{L}(\mathbf{b}', Z)] \geq L_\tau \quad (7)$$

Combining the inequalities, we have shown that for any affine predictor $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$:

$$\mathbb{E}_{\mathbf{X}, Z} [\mathcal{L}(f(\mathbf{X}), Z)] \geq L_\tau \quad (8)$$

Therefore, by Definition 2, $\mathbf{X}$ linearly guards Z with respect to the cross-entropy loss $\mathcal{L}$.

- c)
- d)
- e)
- f)
- g)
- h)
- i)

Question 2: Properties of Finite-state Language Models

a) The Expected String Length

To compute the expected string length μ , we must translate the probability distribution over strings into matrix operations. $\mu \triangleq \sum_{\mathbf{y} \in \Sigma^*} p(\mathbf{y})|\mathbf{y}|$.

In a probabilistic finite-state automaton (PFSA) $\mathcal{A} = (\Sigma, Q, \delta, \lambda, \rho)$, we can group all strings by their length k . The probability of the model generating *any* string of length k is given by the sum of the weights of all paths of length k . Using the transition matrix $\mathbf{T}$, the initial state probabilities λ , and the stopping probabilities ρ , the total probability for all strings of length k is given by $\lambda^\top \mathbf{T}^k \rho$.

$$\mu = \sum_{k=0}^{\infty} k (\lambda^\top \mathbf{T}^k \rho) = \lambda^\top \left(\sum_{k=0}^{\infty} k \mathbf{T}^k \right) \rho \quad (9)$$

To compute this in $\mathcal{O}(|Q|^3)$, we can call the `allsum` algorithm as a black box. The `allsum` algorithm evaluates the sum of weights for all possible paths, computing $\sum_{k=0}^{\infty} \mathbf{W}^k = (\mathbf{I} - \mathbf{W})^{-1}$ for a given weight matrix $\mathbf{W}$.

To make our equation match the `allsum` formulation, we can construct a new augmented automaton using a block matrix structure.

We define a modified PFSA over an expanded state space Q' where each original state is duplicated, meaning $|Q'| = 2|Q|$. This allows the transition matrix to accumulate the path length k as it is raised to higher powers.

We define a new transition block matrix $\mathbf{M} \in \mathbb{R}^{2|Q| \times 2|Q|}$, a new initial state vector $\lambda' \in \mathbb{R}^{2|Q|}$, and a new stopping vector $\rho' \in \mathbb{R}^{2|Q|}$ as follows:

$$\mathbf{M} = \begin{pmatrix} \mathbf{T} & \mathbf{T} \\ \mathbf{0} & \mathbf{T} \end{pmatrix}, \quad \lambda' = \begin{pmatrix} \lambda \\ \mathbf{0} \end{pmatrix}, \quad \rho' = \begin{pmatrix} \mathbf{0} \\ \rho \end{pmatrix} \quad (10)$$

When we raise the block matrix $\mathbf{M}$ to the power of k , the diagonal blocks remain $\mathbf{T}^k$, while the upper-right block accumulates the sum of the products. For any integer $k \geq 0$, we have:

$$\mathbf{M}^k = \begin{pmatrix} \mathbf{T}^k & k \mathbf{T}^k \\ \mathbf{0} & \mathbf{T}^k \end{pmatrix} \quad (11)$$

If we apply the `allsum` algorithm to this new augmented automaton, it computes the sum of the weights over all paths of all lengths:

$$\text{allsum}(\mathbf{M}, \lambda', \rho') = \sum_{k=0}^{\infty} \lambda'^\top \mathbf{M}^k \rho' \quad (12)$$

Substituting block definitions into this equation shows the length coefficient:

$$\lambda'^\top \mathbf{M}^k \rho' = (\lambda^\top \quad \mathbf{0}^\top) \begin{pmatrix} \mathbf{T}^k & k \mathbf{T}^k \\ \mathbf{0} & \mathbf{T}^k \end{pmatrix} \begin{pmatrix} \mathbf{0} \\ \rho \end{pmatrix} \quad (13)$$

$$= (\lambda^\top \mathbf{T}^k \quad \lambda^\top (k \mathbf{T}^k)) \begin{pmatrix} \mathbf{0} \\ \rho \end{pmatrix} \quad (14)$$

$$= \lambda^\top k \mathbf{T}^k \rho \quad (15)$$

Summing this extraction over all $k \in [0, \infty)$ yields the expected length μ :

$$\sum_{k=0}^{\infty} \lambda'^{\top} \mathbf{M}^k \rho' = \sum_{k=0}^{\infty} \lambda^{\top} k \mathbf{T}^k \rho = \mu \quad (16)$$

The Algorithm

1. Construct the block matrix $\mathbf{M}$ alongside vectors λ' and ρ' .
2. Call the standard `allsum` function, passing $(\mathbf{M}, \lambda', \rho')$ as the arguments.
3. Return the scalar output as μ .

Proof of Time Complexity

- Constructing the augmented matrix $\mathbf{M}$ and the vectors involves allocating memory and copying the elements of $\mathbf{T}$, which takes $\mathcal{O}(|Q|^2)$ time.
- The `allsum` algorithm evaluates the infinite sum by calculating the inverse $(\mathbf{I} - \mathbf{M})^{-1}$, or by utilizing Gaussian elimination to solve the equivalent linear system. Matrix inversion and solving linear systems both inherently have a cubic time complexity relative to the size of the matrix.
- Since our augmented matrix $\mathbf{M}$ has dimensions $2|Q| \times 2|Q|$, the `allsum` algorithm will execute in $\mathcal{O}((2|Q|)^3)$ time.
- Because constants can be dropped in Big-O notation, $\mathcal{O}(8|Q|^3)$ simplifies directly to $\mathcal{O}(|Q|^3)$.

The $\mathcal{O}(|Q|^2)$ setup time is strictly bounded and dominated by the $\mathcal{O}(|Q|^3)$ `allsum` operation. Thus, the total overall runtime of the algorithm is proven to be $\mathcal{O}(|Q|^3)$.

Alternative Implementation To elaborate further on the inherent properties of finite-state language models, we can also bypass the block matrix construction to yield a slightly more memory-efficient algorithm that still meets the $\mathcal{O}(|Q|^3)$ requirement. We know from geometric series properties of matrices that $\sum_{k=0}^{\infty} \mathbf{T}^k = (\mathbf{I} - \mathbf{T})^{-1}$. If we evaluate the infinite sum $\sum_{k=0}^{\infty} k \mathbf{T}^k$ by factoring out $\mathbf{T}$ and rewriting it as a matrix derivative, we arrive at:

$$\sum_{k=0}^{\infty} k \mathbf{T}^k = \mathbf{T}(\mathbf{I} - \mathbf{T})^{-2} = (\mathbf{I} - \mathbf{T})^{-1} \mathbf{T} (\mathbf{I} - \mathbf{T})^{-1} \quad (17)$$

This means the mean string length can also be written purely algebraically as $\mu = \lambda^{\top} (\mathbf{I} - \mathbf{T})^{-1} \mathbf{T} (\mathbf{I} - \mathbf{T})^{-1} \rho$. We can compute this directly using a sequence of linear algebraic operations:

1. **Solve** the linear system $(\mathbf{I} - \mathbf{T})\mathbf{v}_1 = \rho$ to find $\mathbf{v}_1$. Time complexity: $\mathcal{O}(|Q|^3)$.
2. **Multiply** the transition matrix to find $\mathbf{v}_2 = \mathbf{T}\mathbf{v}_1$. Time complexity: $\mathcal{O}(|Q|^2)$.
3. **Solve** the second linear system $(\mathbf{I} - \mathbf{T})\mathbf{v}_3 = \mathbf{v}_2$ to find $\mathbf{v}_3$. Time complexity: $\mathcal{O}(|Q|^3)$.
4. **Compute** the final dot product $\mu = \lambda^{\top} \mathbf{v}_3$. Time complexity: $\mathcal{O}(|Q|)$.

The sum of these sequential execution times is $\mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^2) + \mathcal{O}(|Q|^3) + \mathcal{O}(|Q|)$, which perfectly asymptotes to the required $\mathcal{O}(|Q|^3)$ runtime. Both methods are formally correct, though the block matrix approach directly fulfills the hint to deploy the existing `allsum` operation as an undisturbed black box.

b) The Variance of String Lengths

The variance of the string length is defined as $\sigma^2 = \sum_{\mathbf{y} \in \Sigma^*} p(\mathbf{y})(|\mathbf{y}| - \mu)^2$. Using the linearity of expectation:

$$\sigma^2 = \mathbb{E}[|\mathbf{y}|^2] - \mu^2 \quad (18)$$

Sum over all path lengths k to compute the second moment $\mathbb{E}[|\mathbf{y}|^2]$:

$$\mathbb{E}[|\mathbf{y}|^2] = \sum_{k=0}^{\infty} k^2 (\lambda^\top \mathbf{T}^k \rho) = \lambda^\top \left(\sum_{k=0}^{\infty} k^2 \mathbf{T}^k \right) \rho \quad (19)$$

Decompose the k^2 into $k(k-1) + k$ to separate the sum:

$$\sum_{k=0}^{\infty} k^2 \mathbf{T}^k = \sum_{k=0}^{\infty} k(k-1) \mathbf{T}^k + \sum_{k=0}^{\infty} k \mathbf{T}^k \quad (20)$$

These sums map to the derivatives of the geometric matrix series $\sum_{k=0}^{\infty} \mathbf{T}^k = (\mathbf{I} - \mathbf{T})^{-1}$:

1. The first gives the sum for k : $\sum_{k=0}^{\infty} k \mathbf{T}^k = \mathbf{T}(\mathbf{I} - \mathbf{T})^{-2}$.
2. The second gives the sum for $k(k-1)$: $\sum_{k=0}^{\infty} k(k-1) \mathbf{T}^k = 2\mathbf{T}^2(\mathbf{I} - \mathbf{T})^{-3}$.

$$\sum_{k=0}^{\infty} k^2 \mathbf{T}^k = 2\mathbf{T}^2(\mathbf{I} - \mathbf{T})^{-3} + \mathbf{T}(\mathbf{I} - \mathbf{T})^{-2} \quad (21)$$

$$= (\mathbf{I} - \mathbf{T})^{-3} (2\mathbf{T}^2 + \mathbf{T}(\mathbf{I} - \mathbf{T})) = (\mathbf{I} - \mathbf{T})^{-3} (\mathbf{T}^2 + \mathbf{T}) \quad (22)$$

Thus, the second moment is $\mathbb{E}[|\mathbf{y}|^2] = \lambda^\top (\mathbf{T}^2 + \mathbf{T})(\mathbf{I} - \mathbf{T})^{-3} \rho$. Finally, the variance is $\sigma^2 = \lambda^\top (\mathbf{T}^2 + \mathbf{T})(\mathbf{I} - \mathbf{T})^{-3} \rho - \mu^2$.

Algorithm

1. **Compute μ :** Use the algorithm in (a) to find the mean string length.
2. **Initialize:** Define the base vector $\mathbf{v}_0 = \rho$.
3. **Solve First System:** $(\mathbf{I} - \mathbf{T})\mathbf{v}_1 = \mathbf{v}_0$ for $\mathbf{v}_1$. (*This computes $(\mathbf{I} - \mathbf{T})^{-1}\rho$.*)
4. **Solve Second System:** $(\mathbf{I} - \mathbf{T})\mathbf{v}_2 = \mathbf{v}_1$ for $\mathbf{v}_2$. (*This computes $(\mathbf{I} - \mathbf{T})^{-2}\rho$.*)
5. **Solve Third System:** $(\mathbf{I} - \mathbf{T})\mathbf{v}_3 = \mathbf{v}_2$ for $\mathbf{v}_3$. (*This computes $(\mathbf{I} - \mathbf{T})^{-3}\rho$.*)
6. **Compute Polynomial Matrix:** Calculate the matrix $\mathbf{U} = \mathbf{T}^2 + \mathbf{T}$.
7. **Compute Vector:** Calculate the product $\mathbf{w} = \mathbf{U} \mathbf{v}_3$.
8. **Compute Second Moment:** Calculate the dot product $\mathbb{E}[|\mathbf{y}|^2] = \lambda^\top \mathbf{w}$.
9. **Compute Variance:** Return the variance as $\sigma^2 = \mathbb{E}[|\mathbf{y}|^2] - \mu^2$.

Proof of Time Complexity

- **Step 1:** Computing the expected length μ was proven in (a) to take $\mathcal{O}(|Q|^3)$.
- **Steps 3, 4, 5:** Solving a linear system of size $|Q| \times |Q|$ has a worst-case time complexity of $\mathcal{O}(|Q|^3)$. We solve three linear systems sequentially making the time taken is $3 \times \mathcal{O}(|Q|^3)$, which asymptotically is $\mathcal{O}(|Q|^3)$.
- **Step 6:** Multiplying two square matrices of size $|Q| \times |Q|$ to calculate $\mathbf{T}^2$ requires $\mathcal{O}(|Q|^3)$ time. Adding $\mathbf{T}$ requires $\mathcal{O}(|Q|^2)$ additions.

- **Step 7:** Multiplying the $|Q| \times |Q|$ matrix $\mathbf{U}$ by the $|Q| \times 1$ vector $\mathbf{v}_3$ takes $\mathcal{O}(|Q|^2)$.
- **Step 8:** The dot product of two vectors of size $|Q|$ executes in $\mathcal{O}(|Q|)$.
- **Step 9:** A single scalar subtraction operation happens in $\mathcal{O}(1)$ time.

Total sum of the time complexity:

$$\mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^3) + \mathcal{O}(|Q|^2) + \mathcal{O}(|Q|) + \mathcal{O}(1) \quad (23)$$

The highest order term dominates the growth rate, so the algorithm runs in time $\mathcal{O}(|Q|^3)$.

c) Entropy of a Deterministic Finite-State Language Model

Because the automaton $\mathcal{A}$ is deterministic, any string $\mathbf{y}$ corresponds to one unique path of transitions through the automaton from a given starting state.

- Let Π_q be a random variable representing the paths in $\mathcal{A}$ starting in state q .
- Let $\mathbf{H}(\Pi_q)$ denote the entropy of the distribution of paths originating from state q .

Decompose a path Π_q into its first step and the remainder of the path.

- Let τ_q be a random variable representing the outgoing transition from q or its termination.
- Because $\mathcal{A}$ is deterministic, the destination state q' is uniquely determined by τ_q . Consequently, the path Π_q is in bijection with the pair $(\tau_q, \Pi_{q'})$, so their entropies coincide: $\mathbf{H}(\Pi_q) = \mathbf{H}(\tau_q, \Pi_{q'})$.
- By the chain rule of entropy, $\mathbf{H}(x, y) = \mathbf{H}(x) + \mathbf{H}(y \mid x)$. Applying this to the joint variable $(\tau_q, \Pi_{q'})$ yields:

$$\mathbf{H}(\Pi_q) = \mathbf{H}(\tau_q, \Pi_{q'}) = \mathbf{H}(\tau_q) + \mathbf{H}(\Pi_{q'} \mid \tau_q) \quad (24)$$

Evaluate the entropy of the first step, $\mathbf{H}(\tau_q)$. The probability distribution for τ_q is defined taking the transition $q \xrightarrow{y/w} q'$ with w , and taking the termination with ρ_q . The entropy of this single step is the negative sum of $p \log p$ for all possible outcomes:

$$\mathbf{H}(\tau_q) = - \sum_{q \xrightarrow{y/w} q'} w \log w - \rho_q \log \rho_q = \xi_q \quad (25)$$

Evaluate the conditional entropy of the remaining path, $\mathbf{H}(\Pi_{q'} \mid \tau_q)$ represents the expected entropy of the rest of the path, given the first step τ_q . If τ_q is termination with probability ρ_q , the sequence ends, and the entropy of the remaining empty path is 0. If τ_q is a transition to state q' with probability w , the remaining path originates from q' , so its entropy is $\mathbf{H}(\Pi_{q'})$. Summing these expectations over all outgoing transitions gives:

$$\mathbf{H}(\Pi_{q'} \mid \tau_q) = \sum_{q \xrightarrow{y/w} q'} w \mathbf{H}(\Pi_{q'}) \quad (26)$$

We can group this sum by the destination state q' . By definition, the transition matrix entry $\mathbf{T}_{q,q'}$ is the sum of the weights w of all transitions from q to q' . Therefore:

$$\mathbf{H}(\Pi_{q'} \mid \tau_q) = \sum_{q' \in Q} \mathbf{T}_{q,q'} \mathbf{H}(\Pi_{q'}) \quad (27)$$

Construct the matrix equation by substituting the results:

$$\mathbf{H}(\Pi_q) = \xi_q + \sum_{q' \in Q} \mathbf{T}_{q,q'} \mathbf{H}(\Pi_{q'}) \quad (28)$$

To solve, let $\mathbf{H} \in \mathbb{R}^{|Q|}$ be a column vector where the q -th entry is $H(\Pi_q)$. We can represent the system of equations for all states as a single matrix equation:

$$\mathbf{H} = \boldsymbol{\xi} + \mathbf{T} \mathbf{H} \quad (29)$$

Solve for the path entropy vector $\mathbf{H}$ by isolating $\mathbf{H}$ using algebraic matrix operations:

$$\mathbf{H} - \mathbf{T} \mathbf{H} = \boldsymbol{\xi} = (\mathbf{I} - \mathbf{T}) \mathbf{H} \quad (30)$$

Because all eigenvalues of the transition matrix $\mathbf{T}$ have a magnitude less than 1, the matrix $(\mathbf{I} - \mathbf{T})$ is guaranteed to be invertible:

$$\mathbf{H} = (\mathbf{I} - \mathbf{T})^{-1} \boldsymbol{\xi} \quad (31)$$

The overall entropy of the language model $H(p)$ is the expected path entropy weighted by the initial state probabilities. Since $\mathcal{A}$ is deterministic with a unique initial state (so $H(\lambda) = 0$), every string $y \in \Sigma^*$ corresponds to exactly one complete path. The distribution over strings therefore coincides with the distribution over paths, giving $H(p) = H(\Pi) = \sum_{q \in Q} \lambda_q H(\Pi_q)$. Written as the dot product of the initial state distribution and the path entropy vector:

$$H(p) = \lambda^\top \mathbf{H} \quad (32)$$

Substitute the solved vector $\mathbf{H}$ into the final equation:

$$H(p) = \lambda^\top (\mathbf{I} - \mathbf{T})^{-1} \boldsymbol{\xi} \quad (33)$$

Question 3: Efficiently Simulating Bounded Stacks with RNNs

$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_1 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_2$
$\rangle_2$		
$\rangle_1$		
$\langle_2$		

Table 1: The first row of the table represents different stack configurations, while the first column lists the newly read symbols.

a)

b)

$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_1 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_2$
$\rangle_2$		
$\rangle_1$		
$\langle_2$		

Table 2: The first row of the table represents different stack configurations, while the first column lists the newly read symbols.

c)

d)

e)

f)

g)

h)

i)

Question 4: Limitations of Transformers on Simple Languages

- a)
- b)
- c)
- d)
- e)
- f)
- g)
- h)
- i)
- j)
- k)
- l)